TokenBench

Metrics Math

How TPCA, Pareto frontiers, and bootstrap CIs work

A visual, step-by-step explanation of
tokenbench/core/metrics.py

Every formula in plain English.
Worked examples for every edge case.
Flow diagrams tying it all together.

Reads in ~10 minutes. No prior stats knowledge required.

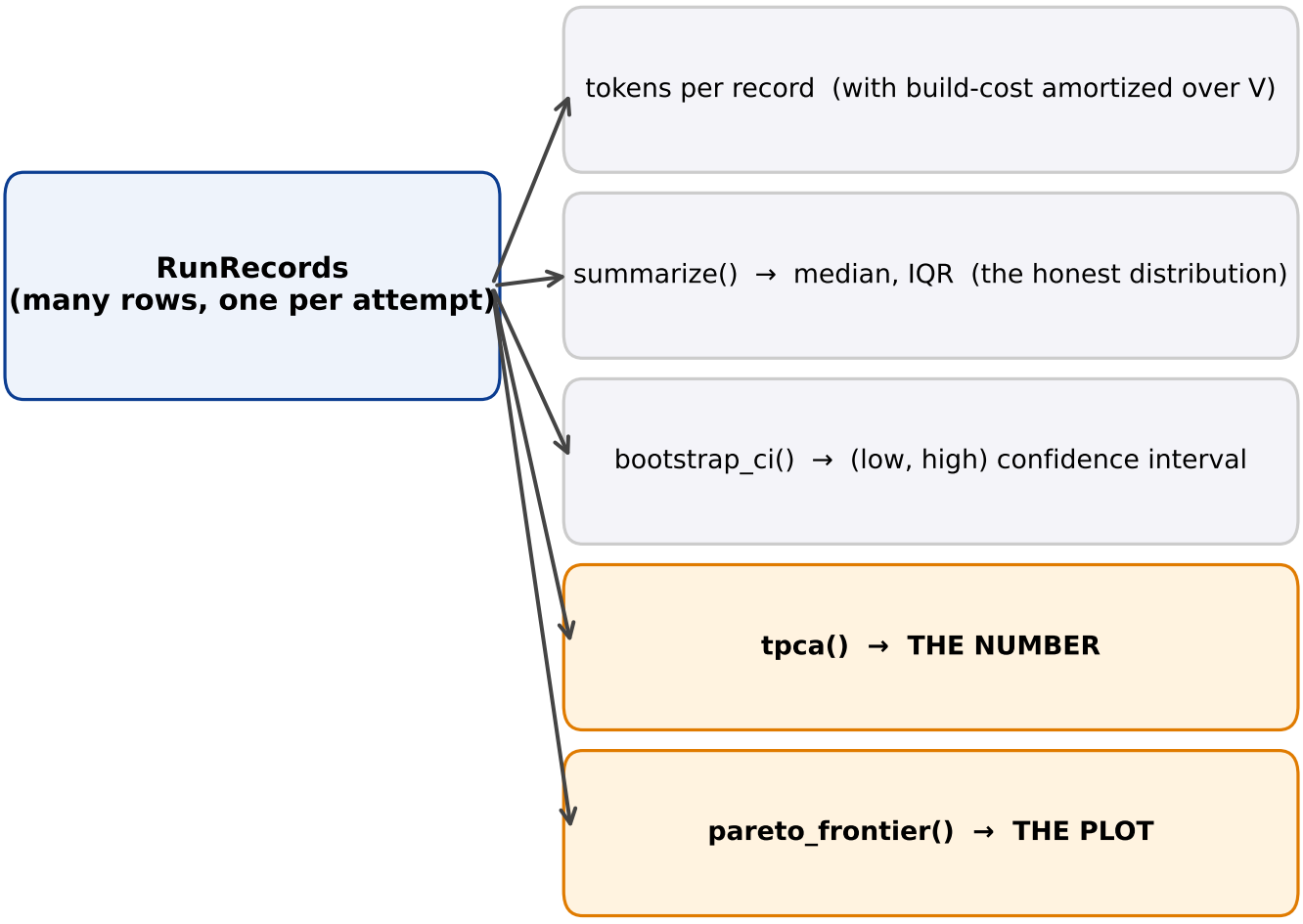
Companion to: tokenbench/core/metrics.py

1. The big picture

What this whole file does, in one diagram.

The benchmark produces a stream of RunRecords — one per attempt.
metrics.py turns those records into three things humans can act on:

- One headline number (TPCA)
- One headline plot (Pareto frontier)
- One uncertainty estimate (bootstrap CI)



Every page that follows zooms into one of these boxes.

2. The building block

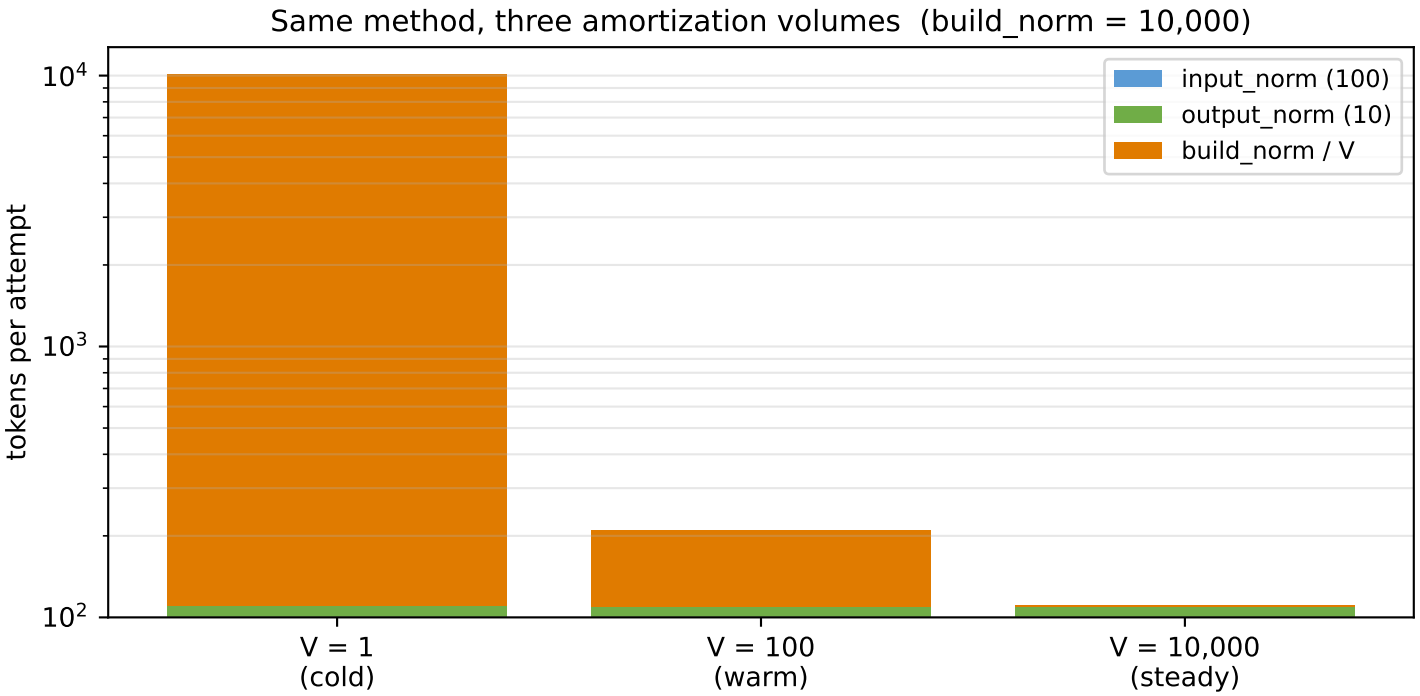
How many tokens does one attempt cost?

Formula

```
tokens(record, V)  =  input_norm  +  output_norm  +  build_norm / V
```

In plain English

Add up: what went IN to the model, what came OUT, plus a fair slice of the one-time setup cost (split across V future queries).



Key insight: build cost dominates at V=1, disappears at V=10,000. This is why the benchmark forbids a single TPCA without stating V.

Edge cases

- `V = 0 or V < 0` → raises ValueError (caught at the boundary)
- `V = ∞` → build_share = 0 (guarded by math.isfinite)
- `V = NaN` → silently treated as ∞ (minor wart, not user-facing)

3. TPCA — the headline number

Tokens Per Correct Answer. Lower is better.

$$\text{TPCA}(V) = \frac{\Sigma \text{ tokens used (all attempts)}}{\Sigma \text{ correct attempts}}$$

In plain English

How many tokens did we burn for every correct answer we got?
WRONG attempts also cost tokens — they sit in the numerator but not the denominator.

Worked example

Attempt	tokens	correct?
1	110	✓
2	110	✓
3	1,100	✗ (wrong but still cost tokens)

$$\Sigma \text{ tokens} = 110 + 110 + 1{,}100 = 1{,}320$$
$$\Sigma \text{ correct} = 2$$
$$\text{TPCA} = 1{,}320 / 2 = 660 \text{ tokens per correct answer}$$

Anti-gaming property

What if a method refuses to answer to avoid being wrong?
→ 0 correct → TPCA = ∞ (worst possible score)
What if a method answers with a single character to save tokens?
→ almost never correct → high tokens per correct, also bad

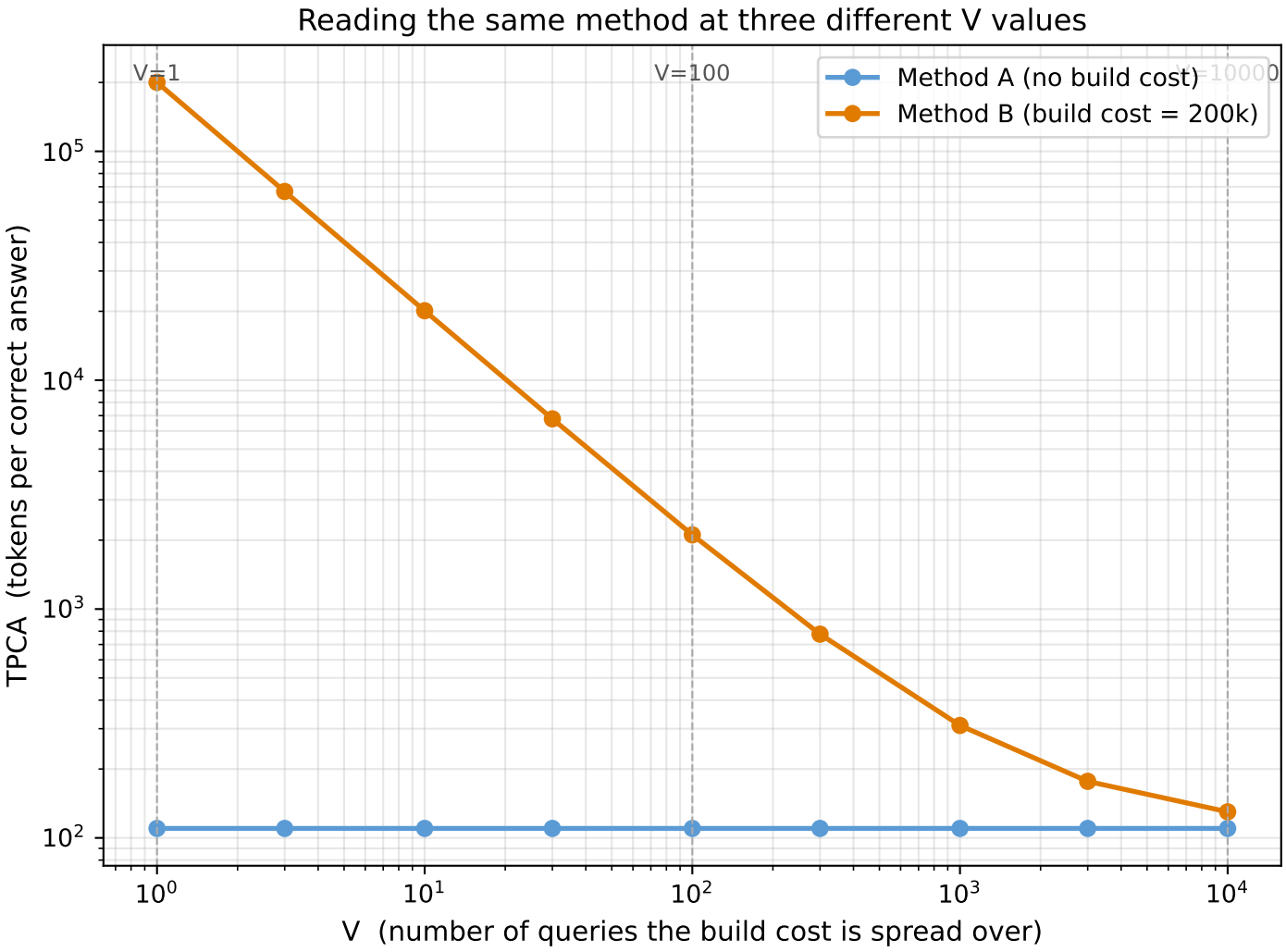
The math punishes both verbosity AND giving up. That's the design.

4. Why V matters

The same method can look terrible OR amazing — depending on V.

Some methods (RAG, Graphify) pay a big one-time setup cost
(building an embeddings index or a code graph).

If you only ask ONE question, that setup cost is wasted.
If you ask 10,000 questions, the setup cost is basically free per query.



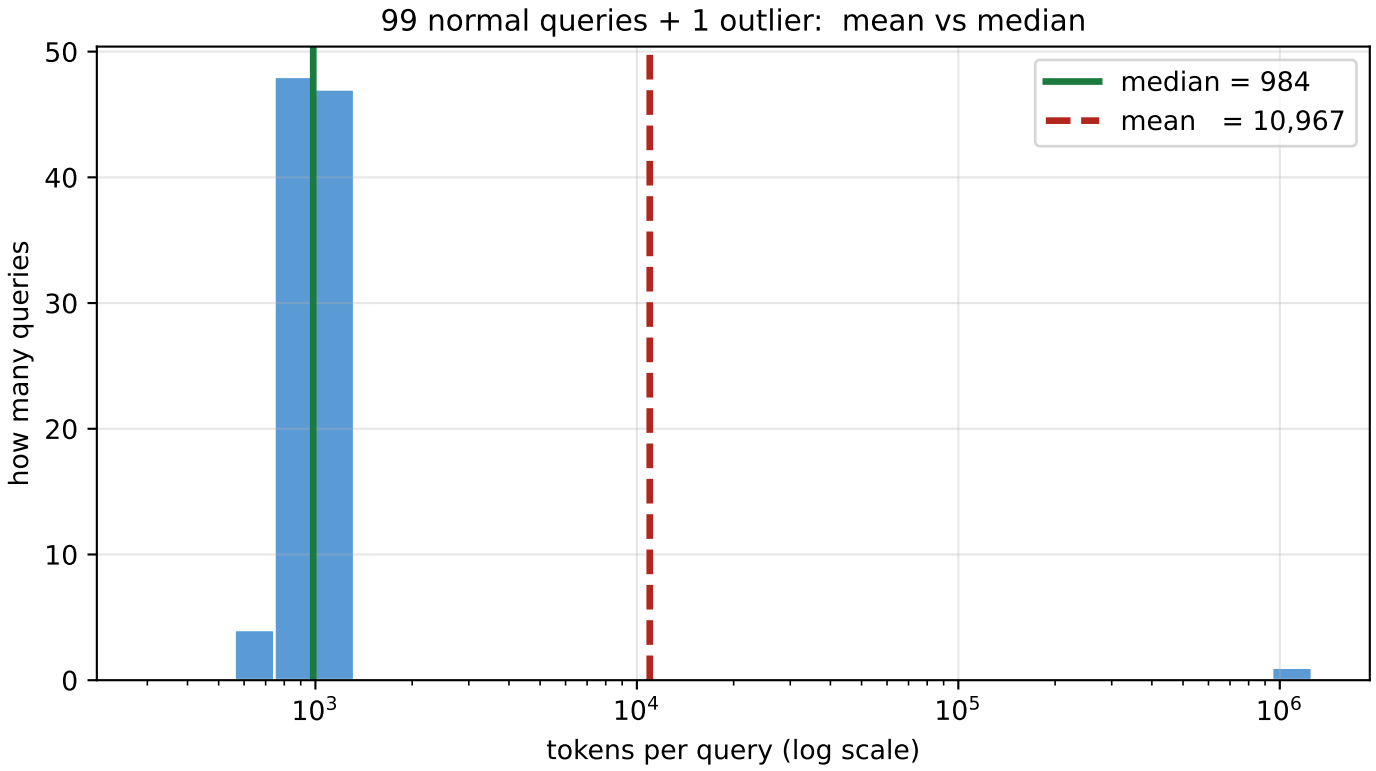

```
At V = 1   (cold start, one query):    Method B looks 1,800× worse than A.
At V = 10,000  (steady state):           Method B looks nearly identical to A.
```

DECISIONS.md #5 → always report TPCA at V ∈ {1, 100, 10,000}.
Reporting a single V is a load-bearing lie.

5. Distribution — why median, not mean

Token usage is heavy-tailed. The mean lies; the median doesn't.

Imagine 100 questions that each used about 1,000 tokens, plus ONE question that hit a pathological case and used 1,000,000 tokens.



Reading this honestly

- `median ≈ 1,000 → 'typical query costs 1,000 tokens' (true)`
- `mean ≈ 11,000 → 'typical query costs 11,000 tokens' (LIE — one outlier did this)`

What summarize() returns

```
Distribution(n, mean, median, q25, q75)

always report median + IQR (q75 − q25) alongside the mean,
never just the mean, per tokenbench_architecture.md §0.1
```

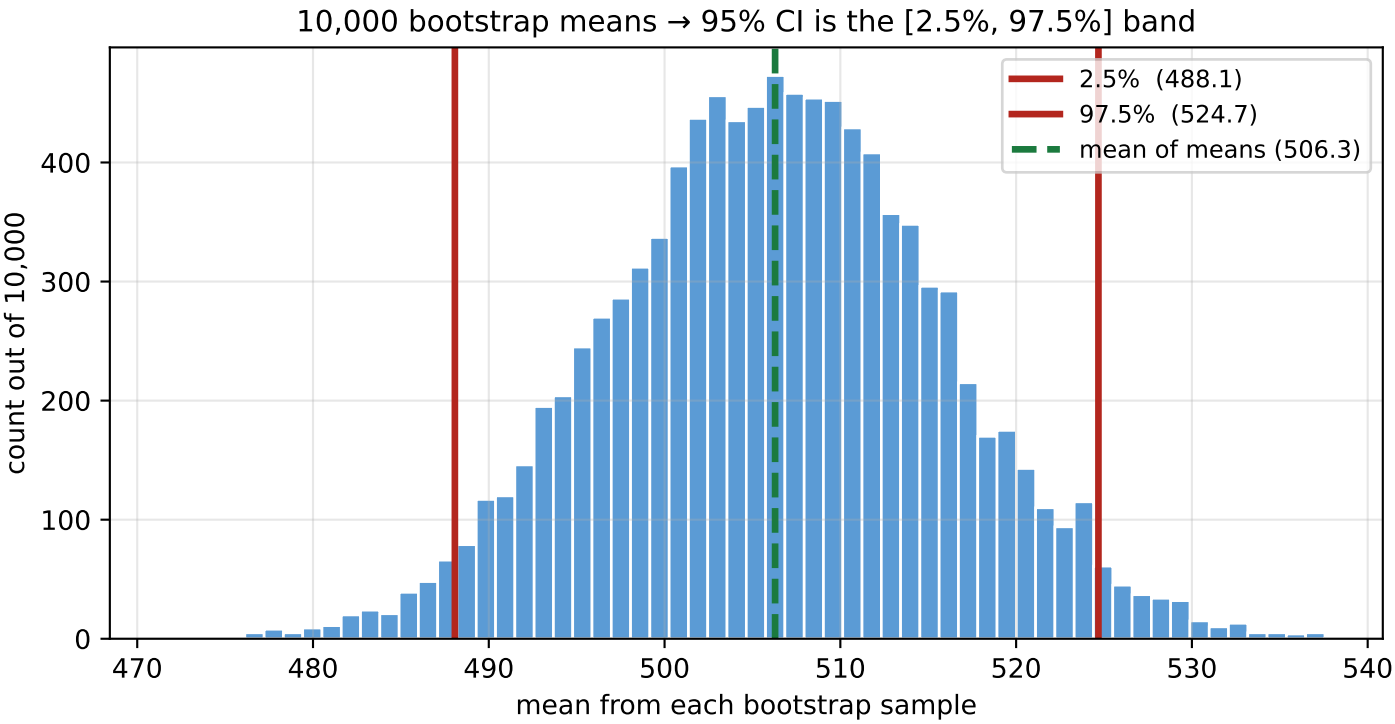
6. Bootstrap CI — how sure are we?

A confidence interval, computed by resampling your own data.

TPCA is a single number — but it has uncertainty.
A confidence interval (CI) says: 'I'm 95% sure the true mean is in this range.'

The algorithm in 4 steps

1. Take your N data points.
2. Resample N points WITH REPLACEMENT → one 'bootstrap sample'.
3. Compute the mean of that sample. Repeat 10,000 times → 10,000 means.
4. The 2.5th and 97.5th percentiles of those means = your 95% CI.



Why this matters when comparing methods

```
If method A is TPCA = 500 [CI: 450, 550]   and
   method B is TPCA = 600 [CI: 580, 620]
   →  CIs don't overlap  →  A is GENUINELY better, not noise.
```

```
If method A is TPCA = 500 [CI: 400, 600]   and
   method B is TPCA = 600 [CI: 500, 700]
   →  CIs overlap  →  CANNOT conclude A is better — likely just noise.
```

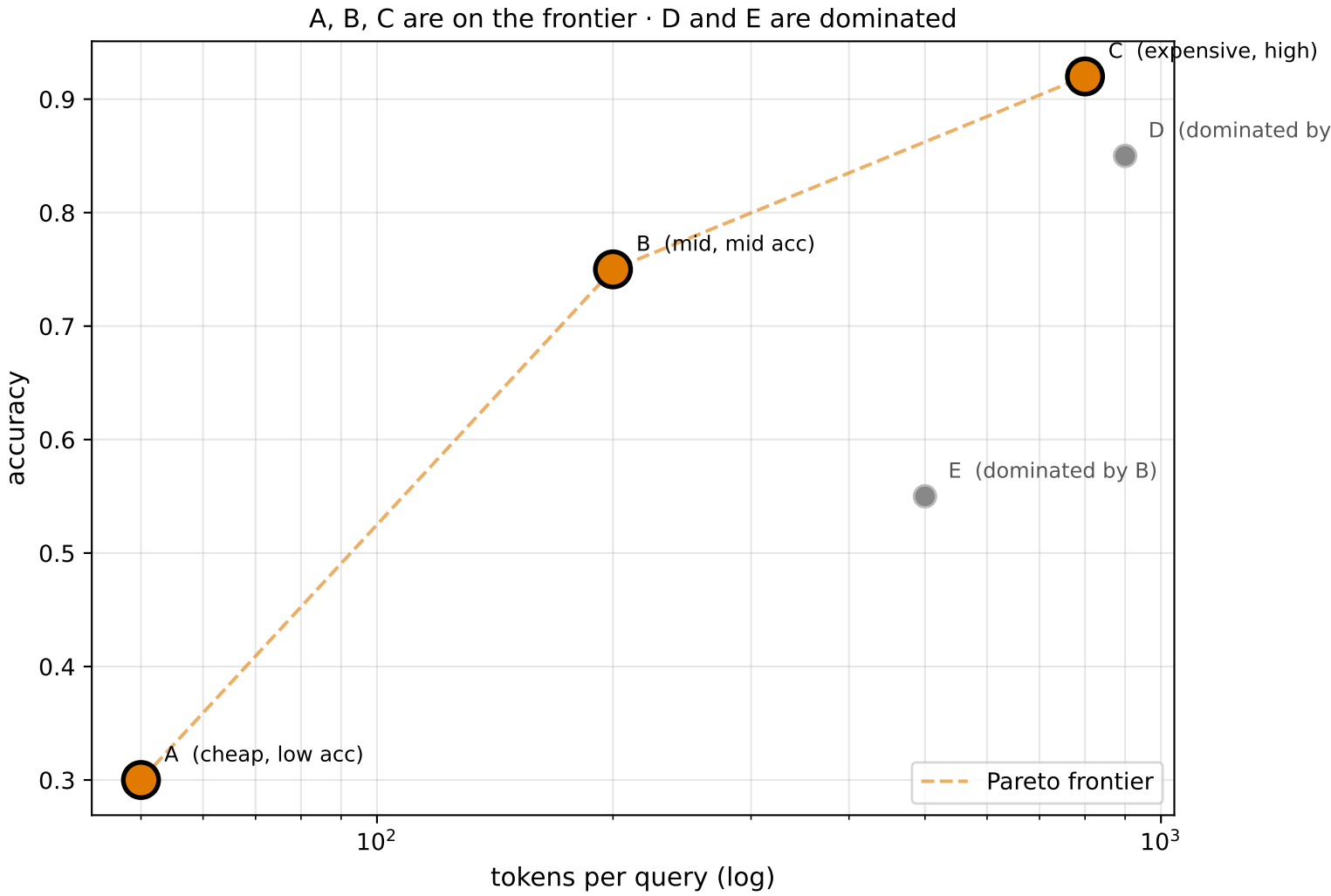
7. Pareto frontier — the headline plot

The set of methods that aren't strictly worse than someone else.

A method is 'dominated' if some other method beats it on BOTH axes
(higher accuracy AND fewer tokens). Dominated methods drop off.

Rule of thumb

> If you ever have to argue your method belongs on the frontier,
> find one axis where nothing else beats you. That's all 'non-dominated' means.

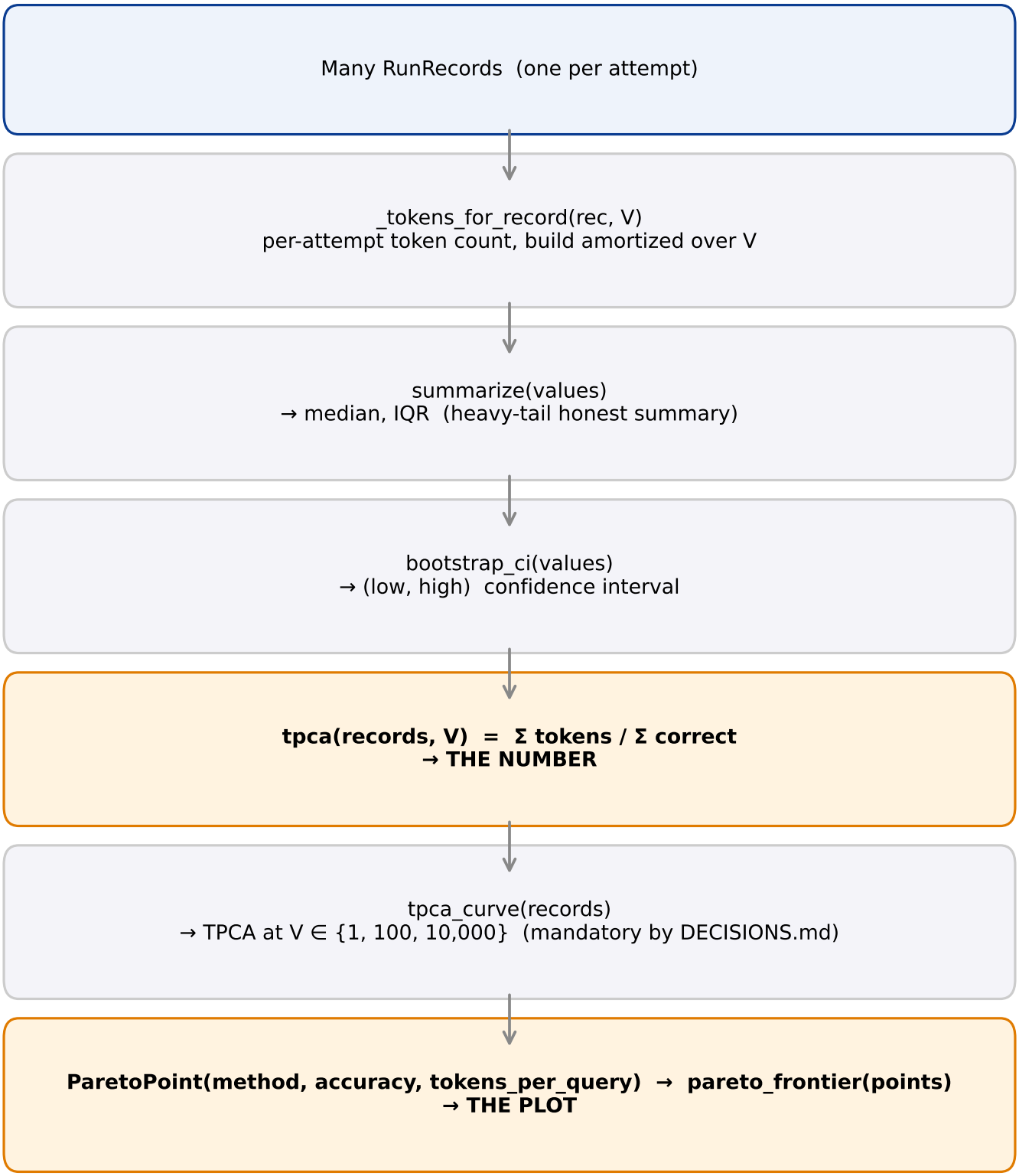


Why this is the headline visual (not a single score)

Different users want different trade-offs: cheap-and-OK vs expensive-and-great.
The frontier shows ALL the legitimate options at once. The score (TPCA) picks one.

8. The complete pipeline

Every function in metrics.py, in the order data flows.



Read the file top-to-bottom and you'll see the functions in roughly this order.
Every later chunk plugs into this same shape — only the data source changes.

9. Cheat sheet

Every formula on one page. Print this and stick it on the wall.

Per-attempt tokens

```
tokens = input_norm + output_norm + (build_norm / V)
```

TPCA — the headline number

```
TPCA(V) = Σ tokens / Σ correct          (lower is better; ∞ if 0 correct)
```

TPCA curve (mandatory for methods with build cost)

```
tpca_curve(records) = { 1: ..., 100: ..., 10000: ... }
```

Accuracy

```
accuracy = Σ correct / N
```

Distribution (heavy-tail aware)

```
summarize(values) → mean, median, q25, q75, n          (report median + IQR)
```

Bootstrap 95% CI on the mean

```
resample 10k times → percentiles[2.5%, 97.5%] of the resample means
```

Pareto frontier

```
drop any point dominated on BOTH axes; keep the rest
```

Companion: tokenbench/core/metrics.py · tests: tests/test_metrics.py

Locked decisions referenced here: DECISIONS.md #1 (tokenizer), #5 (build-cost amortization).